

ENTREGA FINAL DEL TRABAJO PRÁCTICO SOBRE DESARROLLO DE LEXER Y PARSER.

ASIGNATURA: SINTAXIS Y SEMÁNTICA DE LOS LENGUAJES



Cátedra: SINTAXIS Y SEMÁNTICA DE LOS LENGUAJES

Identidad: Utenianos.

Cuatrimestre: I

Carrera: ISI.

Fecha: 06/07/26

INTEGRANTES:

- **Antoniuzzi, Ariadna Abril**
- **Bianchi Julieta Daniela Magali**
- **Gomez, Ricardo Enrique**
- **Martin, Tomás**
- **Medina, Martin Leandro**
- **Zambon Edgar Roman**



-2026-

Índice

Índice	2
Matriz de Habilidades	3
Identidad	3
Alianza de equipo	4
Gramática, documentación técnica	5
Archivos principales del proyecto	5
Análisis léxico	6
Análisis sintáctico	7
Gramática utilizada	7
Análisis semántico	9
Generación de HTML	10
Política de generación de HTML	10
Control de errores	10
Tipos de errores considerados	11
Modo de compilación y ejecución	11
Pruebas realizadas	11
Archivos de prueba utilizados	11
Resultados obtenidos	12
Funciones auxiliares implementadas	12
Conclusión	12
LINK AL VIDEO: https://youtu.be/pnc3sazBbWM	13
Bibliografía / Referencias	13



Matriz de Habilidades

Tras una primera reunión, se trabajó en nueve aspectos fundamentales para el desarrollo de la actividad. Estos incluyen el conocimiento del idioma inglés, habilidades en programación, producción de video y comunicación. Asimismo, se consideraron otros aspectos clave, como el interés por documentar el trabajo, la capacidad de coordinación, la atención a las indicaciones iniciales de los docentes, la resolución de problemas, la innovación y el pensamiento crítico.

Cada integrante del grupo se identificó con al menos uno de estos aspectos, lo que permitió una distribución equitativa de responsabilidades y un trabajo colaborativo eficiente.

<u>Integrantes/ Aptitudes</u>	<u>Inglés</u>	<u>Programa ción</u>	<u>Document ación</u>	<u>Producción de Video</u>	<u>Coordinación</u>	<u>Resolución de Problemas</u>	<u>Innovación</u>	<u>Comunica ción</u>	<u>Pensa mient o crítico</u>
Bianchi Julieta Daniela Magali	o	o	☆	☆	o	☆	☆	o	o
Zambon Edgar Roman	☆			o	o	o	o	☆	o
Medina, Martin Leandro	o	o	o	☆	o	☆		o	☆
Antoniazzi Ariadna Abril	☆		☆	o	o	o		o	o
Ricardo Enrique Gomez	o	o	☆	☆	o		☆	☆	
Martin, Tomás		☆			o	☆	☆		☆

Identidad

(">Utenianos" Que representa el logo breve)

El nombre del grupo, "Utenianos", junto con su logotipo identificador, el cual hace referencia a la institución en la que estudiamos, define un significado claro y consistente: una identidad propia, de carácter académico, representativa para la totalidad de los



integrantes.

La identidad visual ha sido desarrollada bajo un enfoque minimalista, mediante la utilización de una paleta cromática y una simbología específica que estructuran y definen a la gramática. Esta elección responde a la necesidad de garantizar una comunicación efectiva del mensaje, orientado a transmitir valores de pertenencia institucional y rigor académico, a través de una disposición simple y funcional de los elementos (símbolos).

Asimismo, dicha identidad ha sido diseñada con el propósito de asegurar coherencia y claridad en su aplicación, particularmente en el marco del Trabajo Final Integrador, fortaleciendo la vinculación con la institución y consolidando una imagen clara y representativa



Alianza de equipo

Desde la primera reunión del equipo, se estableció como objetivo común la adquisición y consolidación de los conocimientos propios de la asignatura mediante el desarrollo colaborativo del presente trabajo práctico. En este sentido, se identificaron las habilidades, disponibilidades horarias y niveles de compromiso de cada integrante, con el fin de garantizar una distribución equitativa y eficiente de las responsabilidades dentro del grupo.

La metodología de trabajo adoptada consistirá en la realización de reuniones virtuales periódicas, contemplando la posibilidad de encuentros presenciales en caso de resultar necesario. Dichas reuniones se llevarán a cabo durante los fines de semana, en horarios



previamente acordados, utilizando plataformas digitales como Google Meet y Discord, a fin de facilitar la comunicación y el intercambio de ideas.

En cada encuentro, se procederá a la revisión de los objetivos establecidos en la reunión anterior, así como al análisis del progreso alcanzado durante el período de trabajo. Cada integrante expondrá las soluciones desarrolladas y los inconvenientes identificados, promoviendo de este modo el aprendizaje colectivo y la resolución colaborativa de problemáticas.

Asimismo, como herramienta para la gestión y documentación del proyecto, el equipo ha optado por la utilización de Trello, un software de administración de proyectos. A través de esta plataforma, se registran las actividades, los hitos planificados y el avance de cada tarea, lo que permitirá una organización clara, sistemática y accesible para todos los miembros del grupo.

Gramática, documentación técnica

El intérprete SMART-HOME desarrollado por el grupo tiene como objetivo leer archivos con extensión .smart, reconocer sus componentes léxicos, validar su estructura sintáctica, realizar controles semánticos básicos y generar una salida HTML que represente las condiciones y acciones del sistema de domótica.

La implementación final se realizó utilizando Flex para el análisis léxico, Bison para el análisis sintáctico y lenguaje C para las funciones auxiliares, validaciones semánticas y generación del archivo HTML.

El programa final puede ejecutarse desde archivo, recibiendo como argumento un archivo .smart. Esta forma de ejecución fue priorizada porque la entrega final exige que el intérprete pueda analizar archivos de prueba completos y no únicamente entradas escritas manualmente por consola.

Archivos principales del proyecto

Los archivos principales utilizados en la versión final son:

- smart_home_parser.l: archivo del analizador léxico implementado con Flex. Reconoce palabras reservadas, identificadores, operadores, literales, unidades y errores léxicos.
- parser.y: archivo del analizador sintáctico implementado con Bison. Define la gramática del lenguaje, valida la estructura de los programas SMART-HOME, ejecuta validaciones semánticas y registra los elementos necesarios para generar HTML.



- smart_home_parser.exe: ejecutable generado a partir de los archivos parser.y y smart_home_parser.l.
- README_FINAL.md: archivo con instrucciones de compilación, ejecución y pruebas.
- carpeta prueba: contiene archivos .smart válidos e inválidos utilizados para comprobar el funcionamiento del intérprete.

Análisis léxico

El analizador léxico se encarga de leer el archivo de entrada carácter por carácter y agrupar las secuencias reconocidas en tokens. Estos tokens son enviados al parser para que pueda verificar si la estructura general del programa es correcta.

El lexer reconoce las palabras reservadas del lenguaje sin distinguir entre mayúsculas y minúsculas:

WHEN, EVERY, IF, THEN, ELSE, DO, END, AND, OR, NOT.

También reconoce operadores y delimitadores:

==, !=, >, <, >=, <=, =, ., (,).

Además, reconoce identificadores de sensores, dispositivos y atributos. Entre ellos:

- Sensores numéricos: sensor_temp, sensor_temp_int, sensor_humedad, sensor_luz.
- Sensores booleanos: sensor_movimiento, sensor_humo.
- Dispositivos o actuadores: foco_, aire_, persiana_, cerradura_, reloj_, altavoz_, alarma_.
- Atributos: estado, brillo, color, modo, temp_obj, temp_objetivo, temp_act, temp_actual, posicion, hora, fecha, volumen, mute, mensaje, email, email_notif, activada.

También se reconocen literales y unidades:

- Booleanos: TRUE, FALSE, ON, OFF.
- Temperaturas: por ejemplo 22°C o 22C.
- Porcentajes: por ejemplo 80%.
- Tiempo: por ejemplo 30m, 10s, 1h.
- Iluminancia: por ejemplo 250lux.
- Horas: formato HH:MM.
- Fechas: formato DD/MM/AAAA.
- Texto: cadenas entre comillas.
- Emails: direcciones con formato usuario@dominio.extension.

Los comentarios iniciados con // son omitidos para que no interfieran en el análisis sintáctico.



En caso de encontrar un símbolo o unidad no permitida, el lexer informa un error léxico indicando la línea y la cadena cercana al error.

Análisis sintáctico

El análisis sintáctico fue implementado con Bison. Su función es recibir los tokens entregados por el lexer y verificar si están organizados según la gramática del lenguaje SMART-HOME.

La estructura general del programa está compuesta por una o más instrucciones. Las instrucciones admitidas son bloques WHEN, bloques EVERY, condicionales IF y asignaciones.

Gramática utilizada

PROGRAMA → LISTA_INSTRUCCIONES

LISTA_INSTRUCCIONES → INSTRUCCION
| LISTA_INSTRUCCIONES INSTRUCCION

INSTRUCCION → BLOQUE_WHEN
| BLOQUE_EVERY
| BLOQUE_IF
| ASIGNACION

BLOQUE_WHEN → WHEN CONDICION DO LISTA_ACCIONES END

BLOQUE_EVERY → EVERY TIEMPO DO LISTA_ACCIONES END

BLOQUE_IF → IF CONDICION THEN LISTA_ACCIONES END
| IF CONDICION THEN LISTA_ACCIONES ELSE LISTA_ACCIONES END

LISTA_ACCIONES → ACCION
| LISTA_ACCIONES ACCION

ACCION → ASIGNACION
| BLOQUE_IF

ASIGNACION → DISPOSITIVO_ATRIBUTO = VALOR

DISPOSITIVO_ATRIBUTO → DISPOSITIVO . ATRIBUTO

CONDICION → CONDICION OR TERMINO_LOGICO
| TERMINO_LOGICO

TERMINO_LOGICO → TERMINO_LOGICO AND FACTOR_LOGICO
| FACTOR_LOGICO



FACTOR_LOGICO → NOT FACTOR_LOGICO

| (CONDICION)

| COMPARACION

| BOOLEANO

| SENSOR_BOOLEANO

COMPARACION → OPERANDO OPERADOR_COMPARACION OPERANDO

OPERANDO → SENSOR_NUMERICO

| SENSOR_BOOLEANO

| DISPOSITIVO_ATRIBUTO

| VALOR

OPERADOR_COMPARACION → ==

| !=

| >

| <

| >=

| <=

VALOR → BOOLEANO

| MODO

| COLOR

| EMAIL

| TEXTO

| HORA

| FECHA

| TEMPERATURA

| PORCENTAJE

| TIEMPO

| LUX

| NUMERO

| IDENTIFICADOR

Esta gramática permite validar estructuras como:

WHEN sensor_luz < 250lux DO

foco_entrada.estado = ON

END

EVERY 30m DO

IF reloj.hora > 22:00 AND alarma.estado == OFF THEN

persiana_sala.posicion = 0%

END

END



```
IF sensor_humo == TRUE AND aire_acondicionado.estado == OFF THEN
aire_acondicionado.estado = ON
END
```

Cuando el parser encuentra una estructura mal formada, informa un error sintáctico indicando la línea y el lexema cercano al error. En ese caso, no se genera el archivo HTML, ya que no se puede garantizar una traducción correcta de la estructura del archivo fuente.

Análisis semántico

Además del análisis léxico y sintáctico, se implementaron validaciones semánticas básicas. Estas validaciones no revisan solamente si la estructura está bien escrita, sino si los valores utilizados son coherentes con las reglas del lenguaje SMART-HOME.

Entre las validaciones implementadas se encuentran:

- Porcentajes dentro del rango 0% a 100%.
- Valores de iluminancia dentro del rango 0 a 1000 lux.
- Temperaturas de sensores dentro del rango -10°C a 50°C.
- Temperatura objetivo del aire acondicionado dentro del rango 16°C a 30°C.
- Colores válidos para focos: blanco, rojo, azul, blue, red, white.
- Modos válidos para aire acondicionado: FRIO, CALOR, VENT.
- Valores booleanos válidos: TRUE, FALSE, ON, OFF.
- Control de atributos de solo lectura, como reloj.hora, reloj.fecha y valores actuales de sensores o dispositivos que no deberían ser modificados mediante asignación.

Ejemplos de errores semánticos detectados:

```
foco_entrada.brillo = 800%
```

En este caso, la estructura sintáctica es correcta, pero el valor 800% supera el rango permitido de 0% a 100%.

```
aire_acondicionado.temp_obj = 5°C
```

En este caso, la asignación está bien formada, pero la temperatura objetivo está fuera del rango permitido de 16°C a 30°C.

```
reloj_cocina.hora = 22:00
```

En este caso, el atributo hora pertenece a un reloj y se considera de solo lectura, por lo tanto no puede asignarse.

Cuando existen errores semánticos, el programa los informa por pantalla indicando la línea y el lexema cercano al problema. Si se detecta al menos un error semántico, no se genera el archivo HTML, ya que el script no se considera completamente válido.



Generación de HTML

El intérprete genera un archivo HTML a partir del archivo .smart analizado. El nombre del archivo de salida se genera tomando el nombre del archivo fuente y cambiando su extensión por .html.

La traducción a HTML sigue los criterios indicados en la consigna:

- Se genera un dashboard SMART-HOME.
- Las condiciones y sensores se muestran dentro de un bloque con borde verde.
- Los sensores y condiciones se agrupan bajo el título “Estado de sensores y condiciones”.
- Los actuadores se muestran en bloques div con borde gris.
- El nombre de cada actuador se muestra como encabezado.
- Los atributos de cada actuador se traducen como listas ul/li.
- Los valores de tipo email se traducen como enlaces mailto, usando el texto “Contactar a usuario”.

Ejemplo de traducción de email:

altavoz_comedor.email = bomberos@smart-home.com.ar

Se traduce en HTML como un enlace de contacto:

Contactar a bomberos

Política de generación de HTML

Para la generación de HTML se adoptó un criterio estricto: el archivo HTML solo se genera cuando el análisis léxico, sintáctico y semántico finaliza correctamente, sin errores.

Si existen errores léxicos, sintácticos o semánticos, el intérprete informa el tipo de error, el número de línea y el lexema o cadena cercana al problema, pero no genera el archivo HTML. Esta decisión se tomó para asegurar que la salida HTML represente únicamente scripts válidos y correctamente contruidos.

De esta manera, el intérprete diferencia claramente dos situaciones: archivos SMART-HOME válidos que pueden traducirse a HTML y archivos con errores que deben ser corregidos antes de generar una salida.

Control de errores

El intérprete informa los errores indicando:

- Tipo de error.
- Número de línea.



- Cadena o lexema cercano al error.

Tipos de errores considerados

Errores léxicos:

Se producen cuando aparece un símbolo, unidad o secuencia que el lenguaje no reconoce. Por ejemplo, una unidad numérica inválida como 30mm.

Errores sintácticos:

Se producen cuando los tokens reconocidos no respetan la estructura de la gramática. Por ejemplo, un bloque WHEN sin DO o sin END, o una instrucción mal ubicada dentro de un bloque.

Errores semánticos:

Se producen cuando la estructura es correcta pero el valor no cumple las restricciones del lenguaje. Por ejemplo, brillo = 800%, temp_obj = 5°C o asignar un atributo de solo lectura.

Modo de compilación y ejecución

La versión final fue probada en Windows utilizando MSYS2 UCRT64, Flex, Bison y GCC.

Para compilar el proyecto se utilizan los siguientes comandos:

```
bison -d parser.y
flex -o lex.parser.c smart_home_parser.l
gcc -Wall -Wextra -std=c11 parser.tab.c lex.parser.c -o smart_home_parser.exe
```

Para ejecutar el intérprete con un archivo .smart:

```
./smart_home_parser.exe ejemplo_consigna.smart
```

También puede ejecutarse con archivos ubicados en la carpeta prueba:

```
./smart_home_parser.exe "prueba/ejemplo_valido_catedra.smart"
```

Si el análisis es exitoso, el programa informa el éxito y genera el archivo HTML correspondiente. Si existen errores, se informan por pantalla.

Pruebas realizadas

Se realizaron pruebas con archivos válidos e inválidos para verificar el comportamiento del intérprete.

Archivos de prueba utilizados

- ejemplo_consigna.smart: ejemplo propio basado en la consigna.
- ejemplo_valido_catedra.smart: archivo válido proporcionado por la cátedra.



- ejemplo_errores_catedra.smart: archivo con errores provisto por la cátedra.
- error_semantico_puro.smart: archivo creado para probar errores semánticos sin errores léxicos ni sintácticos.
- pruebalexer.smart y pruebalexerinvalidos.smart: archivos utilizados para observar casos de reconocimiento léxico y situaciones inválidas.

Resultados obtenidos

- Con ejemplo_consigna.smart, el análisis léxico, sintáctico y semántico fue exitoso, y se generó el archivo HTML.
- Con ejemplo_valido_catedra.smart, el análisis fue exitoso y se generó el HTML correspondiente.
- Con ejemplo_errores_catedra.smart, el programa detectó errores semánticos, léxicos y sintácticos, informando línea y lexema cercano. Al existir errores léxicos o sintácticos, no se generó HTML final.
- Con error_semantico_puro.smart, el análisis léxico y sintáctico fue exitoso, pero se detectaron errores semánticos. En este caso, el programa informó los errores encontrados y no generó HTML, ya que el archivo no era completamente válido.

Funciones auxiliares implementadas

El archivo parser.y incluye funciones auxiliares para organizar la validación y la generación de salida. Entre ellas se encuentran:

- Funciones para registrar condiciones y asignaciones reconocidas.
- Funciones para validar rangos de porcentajes, temperaturas, lux, horarios y fechas.
- Funciones para comprobar permisos de lectura y escritura de atributos.
- Funciones para detectar atributos de solo lectura.
- Funciones para generar el archivo HTML.
- Funciones para traducir emails a enlaces mailto.
- Funciones para liberar memoria utilizada durante el análisis.
- Funciones para informar errores semánticos, léxicos y sintácticos.
- Funciones para ejecutar el .exe con doble click.

Conclusión

El trabajo permitió integrar los conceptos de análisis léxico, análisis sintáctico y análisis semántico dentro de un intérprete para un lenguaje específico de domótica. A partir de Flex y Bison se implementó un programa capaz de leer archivos .smart, reconocer tokens, validar estructuras, detectar errores y generar una salida HTML representativa del sistema SMART-HOME.

Durante el desarrollo se presentaron dificultades relacionadas con la lectura desde archivo, la integración entre lexer y parser, el control de errores y la generación de HTML. Estas



dificultades fueron resueltas mediante la reorganización del proyecto, la implementación de una versión integrada con Flex/Bison y la incorporación de archivos de prueba.

Como punto fuerte, el intérprete permite analizar archivos completos, informa errores con línea y lexema cercano, genera HTML para archivos válidos y evita generar salida HTML cuando el script presenta errores léxicos, sintácticos o semánticos. Como limitación, algunas validaciones podrían ampliarse en futuras versiones, por ejemplo agregando una verificación más profunda de relaciones entre día, mes y año, o una estructura visual HTML más compleja.

En conclusión, el proyecto cumple con el objetivo principal de construir un intérprete SMART-HOME capaz de analizar, validar y traducir scripts de automatización a una representación HTML.

LINK AL VIDEO: <https://youtu.be/pnc3sazBbWM>

Bibliografía / Referencias

Para el desarrollo del trabajo se utilizaron como referencia los contenidos teóricos de la asignatura Sintaxis y Semántica de los Lenguajes, la consigna oficial del Trabajo Práctico Integrador SMART-HOME y la documentación de las herramientas utilizadas.

Referencias consultadas:

1. Consigna oficial del Trabajo Práctico Integrador: Intérprete SMART-HOME y Traductor a HTML, ciclo lectivo 2026.
2. Material de cátedra de Sintaxis y Semántica de los Lenguajes.
3. Documentación de Flex, herramienta utilizada para el análisis léxico.
4. Documentación de Bison, herramienta utilizada para el análisis sintáctico.
5. Documentación de GCC/MSYS2, entorno utilizado para compilar y probar el intérprete en Windows.